

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО
Научно-учебная лаборатория
методов анализа больших
данных
Стажер-исследователь

УТВЕРЖДАЮ
Академический руководитель
образовательной программы
«Программная инженерия»,
старший преподаватель
департамента программной
инженерии

_____ / М.В. Минец /
«__» _____ 2026 г.

_____ / Н.А. Павлочев /
«__» _____ 2026 г.

**СИСТЕМА РЕГИСТРАЦИИ И УПРАВЛЕНИЯ ЖИЗНЕННЫМ ЦИКЛОМ
МЕРОПРИЯТИЙ**

Программа и методика испытаний

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.09.01-01 51 01–1 ЛУ

Исполнители:

студент группы БПИ238

_____ / М.М. Апаркин /
«__» _____ 2026 г.

студент группы БПИ238

_____ / В. Цуркан /
«__» _____ 2026 г.

студент группы БПИ249

_____ / Г.О. Лещук /
«__» _____ 2026 г.

Москва 2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл.	

УТВЕРЖДЕН

RU.17701729.09.01-01 51 01–1 ЛУ

**СИСТЕМА РЕГИСТРАЦИИ И УПРАВЛЕНИЯ ЖИЗНЕННЫМ ЦИКЛОМ
МЕРОПРИЯТИЙ**

Программа и методика испытаний

RU.17701729.09.01-01 51 01–1

Листов 32

Инов.№ подл.	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

АННОТАЦИЯ

Настоящий документ «Программа и методика испытаний» разработан в соответствии с ГОСТ 19.301–79 и содержит программу и методику проведения испытаний программного изделия «Система регистрации и управления жизненным циклом мероприятий» (CSTATI).

Документ определяет объект и цель испытаний, устанавливает требования к программе, подлежащие проверке, описывает средства и порядок проведения испытаний, а также методы испытаний по каждому проверяемому показателю.

Настоящий документ разработан в соответствии с требованиями:

1. ГОСТ 19.101–77: Виды программ и программных документов.
2. ГОСТ 19.103–77: Обозначения программ и программных документов.
3. ГОСТ 19.104–78: Основные надписи.
4. ГОСТ 19.105–78: Общие требования к программным документам.
5. ГОСТ 19.106–78: Требования к программным документам, выполненным печатным способом.
6. ГОСТ 19.301–79: Программа и методика испытаний. Требования к содержанию и оформлению.
7. ГОСТ 19.603–78: Общие правила внесения изменений.

СОДЕРЖАНИЕ

1. ОБЪЕКТ ИСПЫТАНИЙ	1
1.1. Наименование программы	1
1.2. Область применения	1
1.3. Состав комплекса	1
2. ЦЕЛЬ ИСПЫТАНИЙ	3
3. ТРЕБОВАНИЯ К ПРОГРАММЕ	4
3.1. Функциональные требования	4
3.1.1. Подсистема аутентификации (auth-manager)	4
3.1.2. Подсистема управления мероприятиями (events-service)	4
3.1.3. Подсистема обработки платежей (payment-service)	5
3.1.4. Подсистема уведомлений (notifier)	5
3.1.5. Подсистема Telegram-бота (tg-bot)	5
3.1.6. Подсистема генерации QR-кодов (qr-service)	5
3.1.7. Веб-интерфейс (frontend)	6
3.2. Требования к надёжности	6
3.3. Требования к производительности	6
3.4. Требования к безопасности	6
4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ	8
5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ	9
5.1. Технические средства	9
5.2. Программные средства	9
5.3. Порядок проведения испытаний	10
6. МЕТОДЫ ИСПЫТАНИЙ	11
6.1. Модульное тестирование	11
6.1.1. Метод проведения	11
6.1.2. Проверяемые компоненты	11
6.1.3. Сводные результаты модульного тестирования	14
6.2. Тестирование безопасности	15
6.2.1. Метод проведения	15
6.2.2. Проверяемые показатели	15
6.3. Бенчмарки производительности	16
6.3.1. Метод проведения	16
6.3.2. Результаты бенчмарков Rate Limiting Middleware	17
6.3.3. Анализ результатов	17
6.4. Нагрузочное тестирование	17
6.4.1. Метод проведения	17
6.4.2. Сценарии тестирования (Тест 1 — read path)	18
6.4.3. Профиль нагрузки (Тест 2 — регистрация 100 RPS)	19

6.4.4. Пороговые значения (SLO)	19
6.4.5. Собираемые метрики	19
6.5. Интеграционное тестирование	20
6.5.1. Метод проведения	20
6.5.2. Проверяемые сценарии	20
6.6. Верификация CI/CD	21
6.6.1. Метод проведения	21
6.6.2. Проверяемые показатели	21

1. ОБЪЕКТ ИСПЫТАНИЙ

1.1. Наименование программы

Объектом испытаний является программное изделие «Система регистрации и управления жизненным циклом мероприятий» (далее — «Платформа», «Система», «CSTATI»).

Обозначение программы по ГОСТ 19.103-77: **RU.17701729.09.01-01**.

Вид программного изделия по ГОСТ 19.101-77: **комплекс программ** — совокупность из восьми взаимодействующих программных компонентов.

1.2. Область применения

Платформа CSTATI предназначена для организации и проведения конференций, митапов, хакатонов и иных публичных технических мероприятий с ограниченным числом мест и обязательной онлайн-регистрацией. Система автоматизирует полный жизненный цикл мероприятия: от публикации события и открытия регистрации до подтверждения участия, оплаты, выдачи электронного билета с QR-кодом и контроля прохода на мероприятие.

1.3. Состав комплекса

Комплекс программ включает следующие компоненты:

1. `auth-manager` — микросервис аутентификации и авторизации (Go 1.24);
2. `events-service` — микросервис управления мероприятиями: API-сервер и Kafka-воркер (Go 1.24);
3. `payment-service` — микросервис обработки платежей с интеграцией ЮKassa (Go 1.24);
4. `notifier` — микросервис доставки уведомлений через Telegram (Go 1.24);
5. `qr-service` — микросервис генерации QR-кодов электронных билетов (Go 1.24);
6. `tg-bot` — Telegram-бот для взаимодействия с участниками (Python 3.10, aiogram 3);
7. `frontend` — веб-интерфейс (React 18.3, Vite 5.4);

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

8. `nginx` — обратный прокси и шлюз API (Nginx 1.25).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. ЦЕЛЬ ИСПЫТАНИЙ

Целью испытаний является проверка соответствия разработанного программного изделия «Система регистрации и управления жизненным циклом мероприятий» требованиям, установленным в техническом задании (обозначение: RU.17701729.09.01-01 ТЗ 01–1).

В ходе испытаний проверяются:

1. выполнение функциональных требований к каждой подсистеме комплекса (раздел 4.1 ТЗ);
2. соответствие требованиям к надёжности (раздел 4.2 ТЗ): атомарность операций, механизмы DLQ, идемпотентность, graceful shutdown;
3. соответствие требованиям к производительности (раздел 4.8 ТЗ): время отклика REST API ($p_{99} \leq 500$ мс), пропускная способность Kafka ($\geq 10\,000$ сообщ./мин), 1 000 одновременных соединений;
4. соответствие требованиям к безопасности (раздел 4.8 ТЗ): шифрование TLS, хэширование паролей, JWT-аутентификация, rate limiting, защита от OWASP Top 10;
5. корректность взаимодействия микросервисов через REST API, gRPC и Apache Kafka;
6. работоспособность CI/CD-пайплайна (раздел 4.8 ТЗ).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. ТРЕБОВАНИЯ К ПРОГРАММЕ

Перечень требований, подлежащих проверке во время испытаний, определяется разделом 4 технического задания (RU.17701729.09.01-01 ТЗ 01–1).

3.1. Функциональные требования

3.1.1. Подсистема аутентификации (auth-manager)

1. Регистрация пользователя по адресу электронной почты и паролю (POST /api/v1/auth/sign_up).
2. Аутентификация с выдачей пары JWT-токенов (access: 15 мин, refresh: 30 секунд) через 2FA-верификацию (POST /api/v1/auth/sign_in).
3. Двухфакторная аутентификация: отправка 6-значного кода на email, ограничение 5 попыток ввода, 3 повторные отправки, TTL кода — 5 минут.
4. Обновление access-токена по refresh-токену (POST /api/v1/auth/refresh_token).
5. Межсервисная аутентификация по gRPC (Auth.Authenticate, UserService.GetUser).

3.1.2. Подсистема управления мероприятиями (events-service)

1. CRUD-операции над мероприятиями: создание, просмотр, изменение, фильтрация по статусу, пагинация.
2. Управление волнами регистрации: создание, активация, приостановка, закрытие; автоматическая смена активной волны.
3. Управление билетами: создание с ценой и лимитом мест; атомарное резервирование с оптимистичной блокировкой.
4. Промокоды: создание со скидкой, ограничением использований и сроком действия; активация промокода.
5. Регистрация участников с идемпотентным ключом (кэш Redis, TTL 24 ч).
6. Check-in участника по идентификатору регистрации.
7. Экспорт списка гостей мероприятия.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

8. Kafka-воркеры: `RegistrationWorker` (резервирование мест), `PaymentWorker` (обработка статусов платежей), `ExpirationWorker` (отмена просроченных регистраций каждые 30 с).

3.1.3. Подсистема обработки платежей (`payment-service`)

1. Создание платежа через API ЮKassa с типом подтверждения `redirect`.
2. Обработка бесплатных билетов без обращения к ЮKassa.
3. Приём `webhook`-уведомлений от ЮKassa.
4. Идемпотентность через передачу `registration_id` как ключа идемпотентности.
5. Начисление настраиваемой комиссии (по умолчанию 5 %).

3.1.4. Подсистема уведомлений (`notifier`)

1. Консьюминг уведомлений из Kafka-топика `notifications`.
2. Доставка 5 типов уведомлений через Telegram Bot API: подтверждение регистрации, ссылка на оплату, подтверждение оплаты с QR-кодом, отмена регистрации, 2FA-код.
3. Привязка Telegram-аккаунта (`telegram_username` → `chat_id`).
4. gRPC-интерфейс `NotifierService.SendNotification`.

3.1.5. Подсистема Telegram-бота (`tg-bot`)

1. Обработка команд `/start` и `/help`.
2. Автоматическая привязка `chat_id` при `/start`.
3. Два режима работы: `webhook` и `long-polling`.

3.1.6. Подсистема генерации QR-кодов (`qr-service`)

1. Консьюминг заданий из Kafka-топика `qr-generation`.
2. Генерация PNG-изображения (256×256 пикселей) с идентификатором регистрации.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. Загрузка в MinIO/S3 (bucket `qr-codes`) и публикация URL в `ticket-status-updates`.

3.1.7. Веб-интерфейс (frontend)

1. Публичные страницы: список мероприятий, карточка события, форма регистрации.
2. Административная панель: CRUD мероприятий, управление волнами и билетами, промокоды, экспорт гостей.
3. JWT-аутентификация с автоматическим обновлением access-токена.

3.2. Требования к надёжности

1. Атомарность критических операций с билетами в рамках транзакций PostgreSQL.
2. Dead Letter Queue (DLQ) для необработанных сообщений (5 попыток, экспоненциальная задержка: 1 с, $\times 2$, макс. 30 с).
3. Идемпотентность регистраций (заголовок `Idempotency-Key`, кэш Redis, TTL 24 ч).
4. Идемпотентность платежей (ключ идемпотентности ЮKassa).
5. Graceful shutdown всех сервисов при SIGTERM.
6. Healthcheck-проверки Docker-контейнеров с политикой `restart: always`.

3.3. Требования к производительности

1. Время отклика REST API: $p99 \leq 500$ мс при нагрузке 500 запросов/с.
2. Пропускная способность Kafka: $\geq 10\,000$ сообщений/мин по всем топикам.
3. Одновременные HTTP-соединения: $\geq 1\,000$ без ошибок 5xx.

3.4. Требования к безопасности

1. Шифрование: все внешние эндпоинты доступны только по HTTPS (TLS 1.2+).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. Хэширование паролей: Argon2id (16 МБ, 1 итерация, параллелизм 2).
3. Rate limiting: 100 запросов/с (общий API), 50 запросов/с (эндпоинты аутентификации).
4. Защита внутренних маршрутов HTTP Basic Auth.
5. Защита от SQL-инъекций, XSS, IDOR, открытых редиректов.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

На испытания предъявляется следующий состав программной документации:

1. Техническое задание (ГОСТ 19.201–78) — обозначение:
RU.17701729.09.01-01 ТЗ 01–1.
2. Текст программы (ГОСТ 19.401–78) — обозначение:
RU.17701729.09.01-01 12 01–1.
3. Программа и методика испытаний (ГОСТ 19.301–79) — обозначение:
RU.17701729.09.01-01 51 01–1 (настоящий документ).
4. Пояснительная записка (ГОСТ 19.404–79).
5. Руководство оператора (ГОСТ 19.505–79).

Документация должна быть оформлена в соответствии с ГОСТ 19.106–78.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ

5.1. Технические средства

Испытания проводятся на оборудовании со следующими характеристиками:

Таблица 5.1 — Технические средства проведения испытаний

Параметр	Значение
Процессор	Intel Core i9-12900H (12th Gen), 14 ядер / 20 потоков
Оперативная память	32 ГБ DDR5
Постоянное хранилище	SSD NVMe 512 ГБ
Операционная система	Windows 11 Pro (64-бит)
Docker Engine	24.0+, Docker Desktop for Windows
Go	версия 1.24.0 (windows/amd64)
Python	версия 3.10+
Node.js	версия 20 LTS

5.2. Программные средства

Таблица 5.2 — Программные средства проведения испытаний

Средство	Назначение
<code>go test</code>	Встроенный фреймворк тестирования Go: модульные, интеграционные тесты и бенчмарки
<code>go test -bench</code>	Запуск бенчмарков с измерением производительности (ns/op, B/op, allocs/op)
<code>go test -race</code>	Детектор гонок данных (race detector) для обнаружения конкурентных ошибок
<code>go test -cover</code>	Измерение покрытия кода тестами с генерацией HTML-отчёта
k6	Инструмент нагрузочного тестирования: сценарии с VU, ramp-up, пороговые значения
Go Load Test Tool	Собственная утилита нагрузочного тестирования (cmd/loadtest) с поддержкой конкурентных запросов
<code>golangci-lint</code>	Статический анализ кода Go (линтер)
<code>govulncheck</code>	Проверка зависимостей Go на известные уязвимости

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Средство	Назначение
Docker Compose	Оркестрация контейнеров для интеграционного тестирования
Prometheus + Grafana	Мониторинг метрик при нагрузочном тестировании
Jaeger	Распределённая трассировка запросов при интеграционном тестировании

5.3. Порядок проведения испытаний

Испытания проводятся в следующем порядке:

1. **Модульное тестирование** — проверка корректности работы отдельных компонентов (доменные сущности, сервисы, обработчики, промежуточное ПО, маппинг данных, кэширование). Выполняется с использованием `go test -v`.
2. **Тестирование безопасности** — проверка санитизации ошибок (предотвращение утечки внутренних данных), валидации JWT-токенов, разграничения доступа, rate limiting. Включает статический анализ (`golangci-lint`) и проверку уязвимостей (`govulncheck`).
3. **Бенчмарки производительности** — измерение производительности критических компонентов (rate limiting middleware) с использованием `go test -bench -benchmem`. Проверяется соответствие целевым показателям.
4. **Интеграционное тестирование** — проверка взаимодействия микросервисов в Docker Compose-окружении: сквозные сценарии «регистрация → оплата → уведомление → check-in».
5. **Нагрузочное тестирование** — проверка производительности и устойчивости системы под реальной production-нагрузкой с использованием k6 v0.55.0 против `cstati.com`. Два сценария: многосценарный read path (5 параллельных сценариев, до 229 RPS) и write path (регистрация, до 100 RPS).
6. **Верификация CI/CD-пайплайна** — проверка автоматизированного цикла сборки, тестирования и развёртывания через GitLab CI/CD.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

6. МЕТОДЫ ИСПЫТАНИЙ

6.1. Модульное тестирование

6.1.1. Метод проведения

Модульное тестирование выполняется автоматически с использованием встроенного тестового фреймворка Go (`go test`). Каждый пакет содержит файлы `*_test.go` с тест-функциями, покрывающими основные сценарии работы компонентов. Для изоляции зависимостей используются моки (mock-объекты).

Команда запуска:

```
go test -v -count=1 -short ./internal/...
```

6.1.2. Проверяемые компоненты

6.1.2.1. Доменные сущности (*internal/domain/entities*)

Проверяется корректность создания и валидации доменных объектов:

- создание мероприятия (`TestNewEvent_Success`, `TestNewEvent_IDGeneration`, `TestNewEvent_OwnerIDValidation`);
- вычисление скидок промокодов (`TestPromocode_Calculate*`);
- валидация билетов и волн регистрации (`TestTicket_Validation*`, `TestWave_Validation*`);
- конечные автоматы состояний (`TestRegistration_Fail`, `TestEvent_StatusTransitions`).

Результат: 100 % тестов пройдено успешно, время выполнения — 0,111 с.

6.1.2.2. Сервисный слой (*internal/application/services*)

Проверяется бизнес-логика приложения:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- управление правами владельца мероприятия: TestUpdateEvent_NonAdmin_ReturnsPermissionDenied, TestUpdateEvent_NotOwner_ReturnsNotOwner, TestUpdateEvent_ValidOwner_Success;
- интеграционный сценарий владения: TestEventOwnershipFlow_Integration (5 подтестов: создание администратором, обновление владельцем, отказ не-администратору, отказ не-владельцу, публичный доступ);
- вычисление цен с промокодами: TestPriceCalculator_Calculate_NoPromocode, TestPriceCalculator_Calculate_WithPromocode, TestPriceCalculator_Calculate_WithPromocode_FullDiscount, TestPriceCalculator_Calculate_PromocodeExpired, TestPriceCalculator_Calculate_DiscountCappedAtBasePrice;
- регистрация участников: TestRegister_WaveNotActive_RejectsRegistration, TestRegister_WaveActive_ReservesAndPublishes, TestRegister_TicketsSoldOut, TestRegister_PublishFails_RollsBackReservation, TestRegister_CacheMiss_FallsBackToDB;
- каскадная логика статусов: TestOnTicketSold_WaveSoldOut_ActivatesNextWave, TestOnEventStatusChange_Cancelled_CascadesMultipleWaves, TestAc
- обработка платежей: TestHandlePaymentEvent_Success_*, TestHandlePaymentEvent_WaveSoldOut_AllTicketsSold.

Результат: 100 % тестов пройдено успешно (45 тестов), время выполнения — 0,106 с.

6.1.2.3. HTTP-обработчики (*internal/infrastructure/http/handlers*)

Проверяется корректность обработки HTTP-запросов:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- санитизация ошибок: TestSanitizeError_DomainErrors (4 подтеста), TestSanitizeError_InternalErrors (5 подтестов) — доменные ошибки передаются клиенту, внутренние (SQL, database) скрываются;
- распознавание типов ошибок: TestIsDomainError (5 подтестов), TestIsInternalError (9 подтестов);
- предотвращение утечки SQL: TestSanitizeError_PreventsSQLInjectionLeak;
- авторизация и владение: TestEventHandler_CreateEvent_WithAuth (admin/non-admin/no-token), TestEventHandler_UpdateEvent_OwnershipCheck (owner/non-owner);
- сквозной сценарий авторизации: TestAuthFlow_EndToEnd (3 подтеста);
- идиоматичность регистрации: TestRegisterHandler_Idempotency_CacheHit, TestRegisterHandler_Idempotency_CacheMiss;
- обработка sold-out: TestRegisterHandler_PostRegistrations_TicketsSoldOut (5 подтестов);
- валидация волн: TestWaveHandler_handlePostWaveError (7 подтестов), TestWaveHandler_handleUpdateWaveError (4 подтеста), TestWaveHandler_handleCloseWaveError (3 подтеста).

Результат: 100 % тестов пройдено успешно, время выполнения — 0,119 с.

6.1.2.4. Промежуточное ПО (*internal/infrastructure/http/middlewares*)

Проверяется корректность работы middleware-компонентов:

- аутентификация: TestAuthMiddleware_Authenticate_Success, TestAuthMiddleware_Authenticate_MissingHeader, TestAuthMiddleware_Authenticate_InvalidToken, TestAuthMiddleware_Authenticate_SkipPath;
- проверка администратора: TestAuthMiddleware_RequireAdmin_Success, TestAuthMiddleware_RequireAdmin_NotAdmin;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- правила пропуска: `TestAuthMiddleware_SkipRules_MethodSpecific` (5 подтестов), `TestAuthMiddleware_SkipRules_RegexPattern` (6 подтестов), `TestAuthMiddleware_SkipRules_AnyMethod` (4 подтеста);
- rate limiting: `TestRateLimitMiddleware_AllowsNormalTraffic`, `TestRateLimitMiddleware_BlocksExcessiveRequests`, `TestRateLimitMiddleware_SkipsPaths`, `TestRateLimitMiddleware_DifferentIPsIndependent`, `TestRateLimitMiddleware_RecoveryAfterDelay`;
- проверка утечки горютин: `TestRateLimitMiddleware_NoGoroutineLeak`;
- конкурентный доступ: `TestRateLimitMiddleware_ConcurrentAccess`;
- очистка неактивных лимитеров: `TestRateLimitMiddleware_CleanupRemovesInactiveLimiters`;
- метрики: `TestShouldSkipMetrics` (10 подтестов), `TestNormalizePath` (5 подтестов), `TestIsUUID` (9 подтестов).

Результат: 100 % тестов пройдено успешно, время выполнения — 1,326 с.

6.1.3. Сводные результаты модульного тестирования

Таблица 6.1 — Результаты модульного тестирования

Пакет	Тестов	Результат	Время, с
domain/entities	18	PASS	0,111
application/services/tests	45	PASS	0,106
infrastructure/http/handlers	62	PASS	0,119
infrastructure/http/middlewares	48	PASS	1,326
infrastructure/repositories	15	PASS	0,095
infrastructure/mappers	8	PASS	0,087
infrastructure/pkg/cache	10	PASS	0,102
infrastructure/pkg/circuitbreaker	6	PASS	0,091
Итого	212	PASS	2,037

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

6.2. Тестирование безопасности

6.2.1. Метод проведения

Тестирование безопасности включает автоматизированные проверки в составе модульных тестов, а также статический анализ кода.

6.2.2. Проверяемые показатели

6.2.2.1. Санитизация ошибок (защита от утечки внутренней информации)

Проверяется, что внутренние ошибки (SQL, database, runtime) не передаются клиенту через HTTP-ответы:

- TestSanitizeError_InternalErrors: ошибки типов null pointer, database error, sql error, unknown error заменяются обобщённым сообщением;
- TestSanitizeError_PreventsSQLInjectionLeak: SQL-запросы в тексте ошибок не попадают в ответ;
- TestSanitizeError_PreservesBusinessLogic: доменные ошибки (slug, sold out, wave closed) корректно передаются клиенту.

Результат: все проверки пройдены.

6.2.2.2. Аутентификация и авторизация

- TestAuthMiddleware_Authenticate_MissingHeader: запрос без токена отклоняется;
- TestAuthMiddleware_Authenticate_InvalidToken: невалидный токен отклоняется;
- TestAuthMiddleware_RequireAdmin_NotAdmin: не-администратор не получает доступ к защищённым эндпоинтам;
- TestEventHandler_CreateEvent_WithAuth/Non-admin_token: попытка создания мероприятия не-администратором отклоняется;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- `TestEventHandler_UpdateEvent_OwnershipCheck/Non-owner_admin_cannot_update`: администратор не может изменять чужое мероприятие.

Результат: все проверки пройдены.

6.2.2.3. *Rate Limiting*

- `TestRateLimitMiddleware_BlocksExcessiveRequests`: превышение лимита приводит к ответу HTTP 429;
- `TestRateLimitMiddleware_DifferentIPsIndependent`: лимиты для разных IP-адресов независимы;
- `TestRateLimitMiddleware_RecoveryAfterDelay`: лимит восстанавливается после задержки;
- `TestRateLimitMiddleware_SkipsPaths`: служебные пути (`/health`, `/metrics`) не подлежат ограничению.

Результат: все проверки пройдены.

6.3. Бенчмарки производительности

6.3.1. Метод проведения

Бенчмарки выполняются с использованием `go test -bench -benchmem` на указанном оборудовании (табл. 5.1). Каждый бенчмарк выполняется многократно (до статистически стабильного результата).

Команда запуска:

```
go test -bench=BenchmarkRateLimit -benchmem -run=^$
./internal/infrastructure/http/middlewares/
```

Среда выполнения: Windows, AMD64, Intel Core i9-12900H (12th Gen), 20 потоков.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

6.3.2. Результаты бенчмарков Rate Limiting Middleware

Таблица 6.2 — Результаты бенчмарков rate limiting middleware

Сценарий	Операций	ns/op	B/op	allocs/op
Успешный запрос (в пределах лимита)	2 158 461	531,6	528	8
Отклонённый запрос (превышение лимита)	1 901 754	739,7	528	8
Различные IP-адреса (создание новых лимитеров)	571 737	1 908	6 043	21
Пропуск служебных путей	1 895 887	734,3	1 320	13

6.3.3. Анализ результатов

- **Успешный запрос:** 532 нс на операцию (0,0005 мс) — существенно ниже целевого порога 5 мс.
- **Накладные расходы отклонения:** +208 нс (+39 %) по сравнению с успешным запросом — минимальное влияние на производительность.
- **Различные IP-адреса:** 1,9 мкс — выше из-за создания нового лимитера для каждого IP, но всё равно менее 0,002 мс.
- **Пропуск служебных путей:** 734 нс — быстрый путь для /health и /metrics.

Таблица 6.3 — Соответствие целевым показателям производительности

Компонент	Целевое значение	Фактическое	Статус
Rate limiting middleware	< 5 мс	0,001 мс	Соответствует
API handler (с моками)	< 50 мс	0,01 мс	Соответствует
Операция с Redis	< 5 мс	< 1 мс	Соответствует

6.4. Нагрузочное тестирование

6.4.1. Метод проведения

Нагрузочное тестирование проводится с использованием инструмента **Grafana k6** (версия 0.55.0) непосредственно против production-окружения по адресу `cstati.com`. Сервер: 8 ЦП, 8 ГБ ОЗУ, Яндекс.Облако. Тестирование выполняется с локального клиента через Интернет.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Проводятся два отдельных вида нагрузочного тестирования:

1. **Тест 1: Общий (read path)** — многосценарный тест эндпоинтов чтения с профилем нагрузки, приближённым к реальному трафику. Скрипт: `events-service/tests/load/k6_production_8cpu.js`.
2. **Тест 2: Регистрация 100 RPS (write path)** — стресс-тест эндпоинта `POST /api/v1/registrations` с постепенным выходом на 100 запросов/с. Скрипт: `events-service/tests/load/k6_registration_100rps.js`.

Команды запуска:

```
# Тест 1 — многосценарный (read path)
k6 run --out json=tests/load/results/prod_8cpu_v2.json \
      tests/load/k6_production_8cpu.js

# Тест 2 — регистрация 100 RPS (write path)
k6 run --out json=tests/load/results/registration_100rps.json \
      tests/load/k6_registration_100rps.js
```

6.4.2. Сценарии тестирования (Тест 1 — read path)

Таблица 6.4 — Сценарии нагрузочного тестирования (Тест~1)

Сценарий	Тип нагрузки	Эндпоинт	Целевой RPS	VU макс.
browse_events	ramping-arrival-rate	GET /api/v1/events	150	200
event_detail	ramping-arrival-rate	GET /api/v1/events/{slug}	80	120
waves_and_tickets	constant-arrival-rate	GET /events/{id}/waves + /tickets	30	80
single_ticket	constant-arrival-rate	GET /api/v1/tickets/{id}	15	40
registration	constant-arrival-rate	POST /api/v1/registrations	5	30

Общая длительность: 6 минут (включая ramp-up 90 с и ramp-down 30 с).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

6.4.3. Профиль нагрузки (Тест 2 — регистрация 100 RPS)

Таблица 6.5 — Профиль нагрузки теста регистрации

Этап	Длительность	Целевой RPS
Разогрев	30 с	20
Нарастание	30 с	60
Выход на пик	30 с	100
Удержание пика	2 мин	100
Снижение	30 с	0

Общая длительность: 4 минуты. Максимальное число VU: 300. Payload: POST /api/v1/registrations с уникальным Idempotency-Key (UUID v4) на каждый запрос; Bearer JWT-аутентификация.

6.4.4. Пороговые значения (SLO)

Таблица 6.6 — Пороговые значения нагрузочного тестирования

Метрика	Пороговое значение
p95 латентности GET /events	< 200 мс
p95 латентности GET /events/{slug}	< 300 мс
p95 латентности waves / tickets	< 300 мс
p95 латентности POST /registrations	< 800 мс
p99 латентности (все эндпоинты)	< 2 000 мс
Доля 5xx-ошибок	< 1 %
Успешность (не 5xx)	> 99 %

6.4.5. Собираемые метрики

Для каждого сценария k6 собирает следующие метрики:

- http_req_duration — полное время ответа (мс): min, med, avg, p90, p95, p99, max;
- http_req_failed — доля запросов с ошибкой (сетевой или 5xx);
- http_reqs — общее число запросов и фактический RPS;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- `latency_events_list`, `latency_event_detail`, `latency_waves`, `latency_tickets`, `latency_single_ticket`, `latency_registration` — пользовательские метрики Trend по каждому эндпоинту;
- `registration_error_rate` — доля серверных ошибок (5xx) по эндпоинту регистрации;
- `reg_201_created`, `reg_409_conflict`, `reg_5xx_errors` — счётчики по HTTP-статусам регистрации.

Результаты сохраняются в JSON (`--out json`) и выводятся в консоль в виде сводной таблицы.

6.5. Интеграционное тестирование

6.5.1. Метод проведения

Интеграционные тесты выполняются в полностью развёрнутом Docker Compose-окружении с реальными экземплярами PostgreSQL, Redis, Kafka, MinIO и всех микросервисов. Тесты расположены в каталоге `events-service/tests/integration/`.

Команда запуска:

```
go test -tags=integration -timeout 90s
./tests/integration/...
```

6.5.2. Проверяемые сценарии

6.5.2.1. Сквозной пользовательский сценарий (`user_flow_test.go`)

Проверяется полный цикл работы с системой:

1. Создание мероприятия администратором.
2. Создание волны регистрации и билетной категории.
3. Активация волны.
4. Регистрация участника с указанием email, ФИО и Telegram-аккаунта.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

5. Обработка регистрации Kafka-воркером (RegistrationWorker).
6. Создание платежа и обработка webhook-ответа ЮKassa.
7. Подтверждение оплаты (PaymentWorker).
8. Генерация QR-кода (qr-service).
9. Отправка уведомления с QR-кодом через Telegram (notifier + tg-bot).
10. Check-in участника по идентификатору регистрации.

6.5.2.2. Тест целостности данных (*comprehensive_event_flow_test.go*)

Проверяется корректность каскадных переходов состояний:

1. Автоматическая смена активной волны при sold-out текущей.
2. Деактивация билетов при закрытии волны.
3. Каскадное завершение при отмене мероприятия.
4. Корректность счётчиков мест (reserved, sold) после операций.

6.6. Верификация CI/CD

6.6.1. Метод проведения

Проверяется работоспособность GitLab CI/CD пайплайна, включающего стадии валидации, сборки, развёртывания и верификации.

6.6.2. Проверяемые показатели

1. **Стадия validate:** проверка синтаксиса `docker-compose.yml`, компиляция всех Go-сервисов с генерацией кода (sqlc, oapi-codegen, protoc).
2. **Стадия services-build:** параллельная сборка 9 Docker-образов с помощью Kaniko, публикация в GitLab Container Registry.
3. **Стадия deploy:full:** поэтапный запуск инфраструктуры и сервисов, ожидание статуса healthy (таймаут 180 с).
4. **Стадия verify:** smoke-тесты продакшн-эндпоинтов:
 - GET /health → HTTP 200;
 - GET /api/v1/events → HTTP 200;
 - GET /api/v1/auth/health → HTTP 200;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- GET /api/v1/payment/health → HTTP 200;
- GET /monitoring/grafana/api/health → HTTP 200.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 1

ПРОТОКОЛ МОДУЛЬНОГО ТЕСТИРОВАНИЯ

Ниже приведён фрагмент протокола модульного тестирования events-service, полученный при запуске команды:

```
go test -v -count=1 -short ./internal/...
```

Среда выполнения: Go 1.24.0, Windows/AMD64, Intel Core i9-12900H.

Пакет: domain/entities — доменные сущности:

```
=== RUN    TestNewEvent_Success
--- PASS: TestNewEvent_Success (0.00s)
=== RUN    TestNewEvent_IDGeneration
--- PASS: TestNewEvent_IDGeneration (0.00s)
=== RUN    TestNewEvent_OwnerIDValidation
--- PASS: TestNewEvent_OwnerIDValidation (0.00s)
=== RUN    TestRegistration_Fail
--- PASS: TestRegistration_Fail (0.00s)
ok        domain/entities    0.111s
```

Пакет: application/services/tests — бизнес-логика:

```
=== RUN    TestUpdateEvent_NonAdmin_ReturnsPermissionDenied
--- PASS: TestUpdateEvent_NonAdmin_ReturnsPermissionDenied (0.00s)
=== RUN    TestUpdateEvent_ValidOwner_Success
--- PASS: TestUpdateEvent_ValidOwner_Success (0.00s)
=== RUN    TestEventOwnershipFlow_Integration
--- PASS: TestEventOwnershipFlow_Integration (0.00s)
    --- PASS: .../Admin_owner_can_create_event (0.00s)
    --- PASS: .../Admin_owner_can_update_their_event (0.00s)
    --- PASS: .../Non-admin_cannot_create_event (0.00s)
    --- PASS: .../Admin_cannot_update_other's_event (0.00s)
    --- PASS: .../Public_can_view_events (0.00s)
=== RUN    TestPriceCalculator_Calculate_WithPromocode
--- PASS: TestPriceCalculator_Calculate_WithPromocode (0.00s)
=== RUN    TestPriceCalculator_Calculate_PromocodeExpired
--- PASS: TestPriceCalculator_Calculate_PromocodeExpired (0.00s)
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

=== RUN    TestRegister_WaveActive_ReservesAndPublishes
--- PASS: TestRegister_WaveActive_ReservesAndPublishes (0.03s)
=== RUN    TestRegister_TicketsSoldOut
--- PASS: TestRegister_TicketsSoldOut (0.00s)
=== RUN    TestRegister_PublishFails_RollsBackReservation
--- PASS: TestRegister_PublishFails_RollsBackReservation (0.03s)
=== RUN    TestOnTicketSold_WaveSoldOut_ActivatesNextWave
--- PASS: TestOnTicketSold_WaveSoldOut_ActivatesNextWave (0.00s)
=== RUN    TestHandlePaymentEvent_Success_NoCacheAndNoWaveSoldOut
--- PASS: TestHandlePaymentEvent_Success_NoCacheAndNoWaveSoldOut
(0.00s)
ok    application/services/tests 0.106s

```

Пакет: `infrastructure/http/handlers` — обработчики HTTP:

```

=== RUN    TestSanitizeError_DomainErrors
--- PASS: TestSanitizeError_DomainErrors (0.00s)
    --- PASS: .../Event_slug_error (0.00s)
    --- PASS: .../Tickets_sold_out (0.00s)
    --- PASS: .../Wave_already_closed (0.00s)
=== RUN    TestSanitizeError_InternalErrors
--- PASS: TestSanitizeError_InternalErrors (0.00s)
    --- PASS: .../Database_error_-_should_be_hidden (0.00s)
    --- PASS: .../SQL_error_-_should_be_hidden (0.00s)
=== RUN    TestSanitizeError_PreventsSQLInjectionLeak
--- PASS: TestSanitizeError_PreventsSQLInjectionLeak (0.00s)
=== RUN    TestEventHandler_CreateEvent_WithAuth
--- PASS: TestEventHandler_CreateEvent_WithAuth (0.00s)
    --- PASS: .../Valid_admin_token (0.00s)
    --- PASS: .../Non-admin_token_-_should_fail (0.00s)
    --- PASS: .../No_token_-_should_fail (0.00s)
=== RUN    TestRegisterHandler_Idempotency_CacheHit
--- PASS: TestRegisterHandler_Idempotency_CacheHit (0.00s)
=== RUN    TestRegisterHandler_Idempotency_CacheMiss
--- PASS: TestRegisterHandler_Idempotency_CacheMiss (0.02s)
ok    infrastructure/http/handlers 0.119s

```

Пакет: `infrastructure/http/middlewares` — промежуточное ПО:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

=== RUN    TestAuthMiddleware_Authenticate_Success
--- PASS: TestAuthMiddleware_Authenticate_Success (0.00s)
=== RUN    TestAuthMiddleware_Authenticate_InvalidToken
--- PASS: TestAuthMiddleware_Authenticate_InvalidToken (0.00s)
=== RUN    TestAuthMiddleware_SkipRules_MethodSpecific
--- PASS: TestAuthMiddleware_SkipRules_MethodSpecific (0.00s)
    --- PASS: .../GET_events_should_skip (0.00s)
    --- PASS: .../POST_events_should_require_auth (0.00s)
    --- PASS: .../POST_registrations_should_skip (0.00s)
=== RUN    TestRateLimitMiddleware_NoGoroutineLeak
--- PASS: TestRateLimitMiddleware_NoGoroutineLeak (0.50s)
=== RUN    TestRateLimitMiddleware_ConcurrentAccess
--- PASS: TestRateLimitMiddleware_ConcurrentAccess (0.30s)
=== RUN    TestRateLimitMiddleware_AllowsNormalTraffic
--- PASS: TestRateLimitMiddleware_AllowsNormalTraffic (0.00s)
=== RUN    TestRateLimitMiddleware_BlocksExcessiveRequests
--- PASS: TestRateLimitMiddleware_BlocksExcessiveRequests (0.00s)
ok    infrastructure/http/middlewares 1.326s

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 2

ПРОТОКОЛ БЕНЧМАРКОВ ПРОИЗВОДИТЕЛЬНОСТИ

Ниже приведён протокол бенчмарков rate limiting middleware, полученный при запуске команды:

```
go test -bench=BenchmarkRateLimit -benchmem -run=^$  
./internal/infrastructure/http/middlewares/
```

Среда выполнения: Go 1.24.0, Windows/AMD64, 12th Gen Intel Core i9-12900H, 20 потоков.

Полный вывод утилиты go test:

```
goos: windows  
goarch: amd64  
pkg: gitlab.com/cstati/cstati/event-service/  
      internal/infrastructure/http/middlewares  
cpu: 12th Gen Intel(R) Core(TM) i9-12900H  
  
BenchmarkRateLimitMiddleware_Success-20  
      2158461      531.6 ns/op      528 B/op      8 allocs/op  
BenchmarkRateLimitMiddleware_Rejected-20  
      1901754      739.7 ns/op      528 B/op      8 allocs/op  
BenchmarkRateLimitMiddleware_DifferentIPs-20  
      571737      1908 ns/op      6043 B/op     21 allocs/op  
BenchmarkRateLimitMiddleware_SkipPaths-20  
      1895887      734.3 ns/op     1320 B/op     13 allocs/op  
PASS  
ok   middlewares  7.833s
```

Пояснение столбцов:

- **Операций** — количество итераций, выполненных бенчмарком для получения статистически устойчивого результата;
- **ns/op** — среднее время одной операции в наносекундах;
- **B/op** — среднее количество байт, выделенных на одну операцию;
- **allocs/op** — среднее количество аллокаций памяти на одну операцию.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 3

КОНФИГУРАЦИЯ НАГРУЗОЧНОГО ТЕСТИРОВАНИЯ K6

Ниже приведены ключевые фрагменты конфигурации k6 для двух видов нагрузочного тестирования.

Тест 1 — многосценарный (read path), файл k6_production_8cpu.js:

```
export const options = {
  scenarios: {
    browse_events: {
      executor: 'ramping-arrival-rate',
      startRate: 0, timeUnit: '1s',
      preAllocatedVUs: 100, maxVUs: 200,
      stages: [
        { duration: '30s', target: 30 },
        { duration: '60s', target: 100 },
        { duration: '60s', target: 150 },
        { duration: '3m', target: 150 },
        { duration: '30s', target: 0 },
      ],
      exec: 'scenBrowseEvents',
    },
    event_detail: { /* 80 RPS max, ramping-arrival-rate */ },
    waves_and_tickets: { executor: 'constant-arrival-rate', rate:
30 },
    single_ticket: { executor: 'constant-arrival-rate', rate:
15 },
    registration: { executor: 'constant-arrival-rate', rate:
5 },
  },
  thresholds: {
    latency_events_list: ['p(95)<200', 'p(99)<500'],
    latency_event_detail: ['p(95)<300', 'p(99)<700'],
    latency_registration: ['p(95)<800', 'p(99)<2000'],
    http_req_failed: ['rate<0.01'],
  },
};
```

Тест 2 — регистрация 100 RPS, файл k6_registration_100rps.js:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

export const options = {
  scenarios: {
    registration_100rps: {
      executor: 'ramping-arrival-rate',
      startRate: 0, timeUnit: '1s',
      preAllocatedVUs: 150, maxVUs: 300,
      stages: [
        { duration: '30s', target: 20 },
        { duration: '30s', target: 60 },
        { duration: '30s', target: 100 },
        { duration: '2m', target: 100 },
        { duration: '30s', target: 0 },
      ],
    },
  },
  thresholds: {
    registration_latency_ms: ['p(95)<800', 'p(99)<2000',
    'med<200'],
    registration_error_rate: ['rate<0.01'],
    registration_success_rate: ['rate>0.99'],
  },
};

// Payload с уникальным Idempotency-Key на каждый запрос:
export default function () {
  const res = http.post(`${API}/registrations`,
    JSON.stringify({ fio, email, tg, ticket_id: TICKET_ID,
      payment_method: 'online' })),
    { headers: {
      'Authorization': `Bearer ${JWT}`,
      'Idempotency-Key': uuid(), // новый UUID на каждый запрос
    }
  }
};
}

```

—

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 4

ПРОТОКОЛ НАГРУЗОЧНОГО ТЕСТИРОВАНИЯ (PRODUCTION)

Дата проведения: 21 апреля 2026 г. Цель: <https://cstat1.com>. Сервер: 8 ЦП, 8 ГБ ОЗУ, Яндекс.Облако.

Тест 1 — многосценарный (read path):

Строка запуска:

```
k6 run --out json=tests/load/results/prod_8cpu_v2.json \
  tests/load/k6_production_8cpu.js
```

Сводный вывод k6:

```
running (6m00.2s), 000/235 VUs, 74400 complete and 0 interrupted
iterations

browse_events      [=====] 000/100 VUs 6m0s
event_detail      [=====] 000/060 VUs 5m0s
single_ticket     [=====] 00/20 VUs 5m0s
waves_and_tickets [=====] 00/40 VUs 5m0s
registration       [=====] 00/15 VUs 3m0s

http_reqs.....: 82504 229.06/s
http_req_duration (all):
  med=7.5ms p90=10.82ms p95=17.21ms p99=58.22ms max=337.81ms

latency_events_list p95=15.29ms p99=54.25ms count=41099
latency_event_detail p95=17.73ms p99=75.52ms count=18899
latency_waves       p95=19.62ms p99=69.56ms count=9001
latency_tickets     p95=17.85ms p99=33.88ms count=9001
latency_single_ticket p95=16.25ms p99=71.62ms count=4501

http_req_failed: 0.09% (82 из 82504 — TCP-таймауты, не 5xx)
error_rate:      0.09%
success_rate:    99.90%
```

Тест 2 — регистрация 100 RPS (write path):

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Строка запуска:

```
k6 run --out json=tests/load/results/registration_100rps.json \
  tests/load/k6_registration_100rps.js
```

Сводный вывод k6:

```
running (4m00.0s), 000/150 VUs, 17399 complete and 0 interrupted
iterations

http_reqs.....: 17399 72.50/s
reg_201_created: 17277 (99.3%)
reg_5xx_errors: 0 (0.0%)

registration_latency_ms (2xx only):
  med=51.21ms p90=67.03ms p95=75.52ms p99=174.20ms max=14.59s*

registration_error_rate: 0.00% (серверных ошибок 5xx нет)
registration_success_rate: 100.00%

* max 14.59s — единственный outlier (TCP-retry на клиенте), не 5xx
```

Примечание: фактический RPS 72,5 (вместо целевых 100) обусловлен ограничениями сетевого стека WSL2 на клиентской машине. Серверных ошибок (5xx) — 0 при любой нагрузке.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 4**СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ**

1. ГОСТ 19.101–77 Виды программ и программных документов // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.103–77 Обозначения программ и программных документов // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.104–78 Основные надписи // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.105–78 Общие требования к программным документам // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.106–78 Требования к программным документам, выполненным печатным способом // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.201–78 Техническое задание. Требования к содержанию и оформлению // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.301–79 Программа и методика испытаний. Требования к содержанию и оформлению // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.603–78 Общие правила внесения изменений // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]